“Code Mitra”

A Major Project Report Submitted to
Rajiv Gandhi Proudyogiki Vishwavidyalaya

Towards Partial Fulfillment for the Award of
Bachelor of Technology in
CSE(Artificial Intelligence & Machine Learning)

Guided By

Prof. Sumit Jain
(Assistant Professor)

Submitted By

Rishiraj Patidar (0827AL221111)
Soumay Choudhary (0827AL221131)
Koushal Sharma (0827AL233D05)

Acropolis Institute Of Technology and Research, Indore

2025-2026

Declaration

I hereby declared that the work, which is being presented in the project entitled **Code Mitra** partial fulfilment of the requirement for the award of the degree of **Bachelor of Technology**, submitted in the department of CSE (Artificial Intelligence & Machine Learning) at **Acropolis Institute of Technology and Research, Indore** is an authentic record of my own work carried under the supervision of "**Prof. Sumit Jain**". I have not submitted the matter embodied in this report for award of any other degree.

Rishiraj Patidar (0827AL221111)
Soumay Choudhary (0827AL221131)
Koushal Sharma (0827AL233D05)

Prof. Sumit Jain

Project Guide

Project Approval Form

I hereby recommend that the project **Code Mitra** prepared under my supervision by **Rishiraj Patidar (0827AL221111) Soumay Choudhary (0827AL221131) Koushal Sharma (0827AL233D05)** be accepted in partial fulfillment of the requirement for the degree of Bachelor of Technology in CSE (Artificial Intelligence & Machine Learning).

Prof. Sumit Jain

Project Guide

Recommendation concurred in 2025-2026

Dr. Mayur Rathi

Prof. Sumit Jain

Project Coordinator

Acropolis Institute of Technology and Research

Deparment of CSE(Artificial Intelligence & Machine Learning)

Certificate

The project work entitled **Code-Mitra** submitted by **Rishiraj Patidar (0827AL221111) Soumay Choudhary (0827AL221131) Koushal Sharma (0827AL233D05)** is approved as partial fulfillment for the award of the degree of Bachelor of Technology in CSE(Artificial Intelligence & Machine Learning) by Rajiv Gandhi Proudyogiki Vishwavidyalaya, Bhopal (M.P.)

Internal Examiner

Name:…….

Date: …./…/……..

External Examiner

Name: ……..

Date: …./…/……..

Acknowledgement

With boundless love and appreciation, we/I would like to extend our/my heartfelt gratitude and appreciation to the people who helped us/me to bring this work in reality. We/I would like to have some space of acknowledgement for them.

Foremost, our/I would like to express our/ my sincere gratitude to our/my supervisor, **Prof. Sumit Jain** whose expertise, consistent guidance, ample time spent and consistent advices that helped us/me to bring this study into success.

To the project coordinator **Dr. Mayur Rathi & Prof. Sumit Jain** for their constructive comments, suggestions, and critiquing even in hardship.

To the honourable **Dr. Namrata Tapaswi**, Head, Department of CSE(Artificial Intelligence & Machine Learning) for his favourable responses regarding the study and providing necessary facility.

To the honourable **Dr. S.C. Sharma**, Director, for his unending support, advises and effort to make possible.

Finally, I/we would like to pay my/our thanks to faculty members and staff of Department of CSE(Artificial Intelligence & Machine Learning) for their timely help and support.

We/I also like to pay thanks to our/my **parents** for their eternal love, support and prayers.Without them it is not possible.

Rishiraj Patidar (0827AL221111)
Soumay Choudhary (0827AL221131)
Koushal Sharma (0827AL233D05)

Abstract

CODE MITRA is currently being built as an advanced, student-centric coding education platform that is transforming the way learners are understanding and practicing programming. Instead of emphasizing only the correctness of the final output, as many traditional coding platforms do, CODE MITRA is focusing on strengthening logic-building, conceptual clarity, and pattern recognition. The project is addressing major issues such as over-reliance on copied solutions, weak reasoning skills, and limited exposure to multiple approaches for solving a single problem. By guiding learners through brute-force solutions, intermediate optimizations, and ultimately optimal techniques, the platform is nurturing structured thinking and long-term problem-solving capability. Through this pattern-oriented and strategy-driven methodology, the system is encouraging students to explore the "why" behind every solution rather than blindly memorizing code.

To achieve these objectives, CODE MITRA is integrating a powerful combination of modern frameworks, cloud-based services, and intelligent AI components. The system architecture is being built with React/Next.js on the frontend and Node.js/Spring Boot on the backend, supported by secure SQL databases for reliable data handling. Real-time code execution is being enabled through the Piston API, while AI/ML models are continuously generating contextual hints, explaining errors, analyzing user progress, and offering guided feedback. Alongside learning tools, a secure institutional testing module with live proctoring, plagiarism detection, and automated evaluation is being developed to support universities and colleges in conducting fair and efficient assessments. Gamification features such as badges, streaks, scorecards, and pattern-based progress indicators are being added to maintain motivation and make learning more enjoyable.

As development continues, CODE MITRA is consistently demonstrating improvements in how learners approach coding challenges, think logically, and correct their mistakes independently. Personalized analytics are helping students identify their strengths and weaknesses, enabling targeted growth and continuous improvement. The platform is showing strong potential to reduce academic dishonesty, increase genuine learning efforts, and prepare learners more effectively for technical interviews and real-world software tasks. With its emphasis on ethical learning, structured guidance, and scalable architecture, CODE MITRA is emerging as a transformative ecosystem that is reshaping the future of coding education.

Table of Content

Declaration **1**

Project Approval Form **2**

Acknowledgement **3**

Abstract **4**

Table of Content **5**

List of Figures **7**

List of Tables **7**

Abbreviations **9**

Chapter 1: Introduction

1.1 Rationale **15**

1.2 Existing System 15

1.3 Problem Formulation 15

1.4 Proposed System 15

1.5 Objectives 15

1.6 Contribution of the Project 16

1.6.1 Market Potential 16

1.6.2 Innovativeness 16

1.6.3 Usefulness 16

1.7 Report Organization 16

Chapter 2: Review of Literature

2.1 Preliminary Investigation 18

2.1.1 Current System 18

2.2 Limitations of Current System 19

2.3 Chapter Summary 19

Chapter 3: Requirement Engineering

3.1 Feasiblity Study (Technical, Economical, Operational) 20

3.2 Requirement Collection 20

3.2.1 Discussion 20

3.2.2 Requirement Analysis 20

3.3 Requirements 21

3.3.1 Functional Requirements 21

3.3.1.1 Statement of Functionality 21

3.3.2 Nonfunctional Requirments 21

3.3.2.1 Statement of Functionality 21

3.4 Hardware & Software Requirements 22

3.4.1 Hardware Requirement (Developer & End User) 22

3.4.2 Software Requirement (Developer & End User) 22

3.5 Use-case Diagrams 23

3.5.1 Use-Case Descriptions 23

Chapter 4: Analysis & Conceptual Design & Technical Architecture

4.1 Technical Architecture 24

4.2 Sequence Diagrams 25

4.3 Class Diagrams 26

4.4 DFD 27

4.5 User Interface Design 28

4.6 Data Design 29

4.6.1 Schema Definitions 30

4.6.2 E-R Diagram 30

5. Conclusion & Future Scope

5.1 Conclusion 31

5.2 Future Scope 31

REFERENCES 32

List of Figures

Figure 3.1: Use case diagram 23

Figure 4.1: Technical architecture 24

Figure 4.2: Sequence diagram 25

Figure 4.3: Class diagram 26

Figure 4.4: DFD diagram 27

Figure 4.5.1.: Interface design 28

Figure 4.5.2: Interface Diagram 29

Figure 4.6:ER diagram 30

List of Tables

Table 2.3 : Chapter summary 19

Table 3.3.1.1: Functional requirement 21

Table 3.3.2.1: Non Functional requirement 21

Table 3.4.1: Hardwere requirement 22

Table 3.4.2: softwere requirement 22

Table 4.6.1: Schema Defination 29

Abbreviations

Abbr1: *AI* – Artificial Intelligence

Abbr2: *IDE* - Integrated Development Environment

Abbr3: *UI* - User Interface

Abbr4: *UX* – User Experience

Abbr5: *API* - Application Programming Interface

Abbr6: *DBMS* - Database Management System

Abbr7: *SQL* - Structured Query Language

Abbr8: *LLM* - Large Language Model

Abbr9: *VC* - Version Control

Abbr10: *CI/CD* - Continuous Integration / Continuous Deployment

Abbr11: *SDK* - Software Development Kit

Abbr12: *Grok AI* - Grok Artificial Intelligence

Abbr13: *SRC* - Software Requirement Specification

INTRODUCTION

1.1 Rationale

Modern coding education requires more than correctness checking — it must teach patterns, debugging strategies, and reasoning. CodeMitra aims to address these needs by combining AI guidance, pattern-centric content, and secure institutional assessment.

1.2 Existing System

Major online platforms (LeetCode, HackerRank) focus on test-case correctness and problem difficulty. They provide practice and forums but do not emphasize stepwise learning paths (brute→optimal) or delayed, contextual guidance that encourages independent thinking. Institutional testing often relies on manual evaluation or simple automated checks vulnerable to plagiarism.

1.3 Problem Formulation

Students often resort to copying solutions or trial-and-error to pass test cases, leading to weak conceptual understanding and poor readiness for interviews. Institutions face challenges ensuring fair assessments and providing meaningful feedback at scale.

1.4 Proposed System

CodeMitra is an AI-powered learning and assessment platform that:

(a) organizes problems by fundamental patterns .

(b) enforces progressive solution learning (Brute → Improved → Optimal) .

(c) provides time-gated contextual hints and AI debugging .

(d) implements anti-copy and proctoring measures for institutional tests .

(e) offers gamified performance tracking.

1.5 Objectives

- Design a pattern-based problem repository and tagging system.
- Implement an AI coach to generate contextual hints and explain errors.
- Build a secure institutional testing module with anti-cheating measures.
- Integrate real-time code execution and an explainable debugger.
- Provide gamified scorecards and analytics for learners and educators.

1.6 Contribution of the Project

1.6.1 Market Potential

There is a growing need among universities, coding bootcamps, and individual learners for systems that effectively combine teaching, practice, and evaluation. Existing platforms either focus on learning (tutorial-based) or assessment (exam-based), but rarely offer both in an integrated form. CodeMitra fills this market gap by offering a complete ecosystem for structured coding education and secure institutional deployment. Its scalability and adaptability make it ideal for integration into formal curricula as well as self-paced learning.

1.6.2 Innovativeness

CodeMitra's innovation lies in uniting AI tutoring, pattern-centric pedagogy, and secure assessment within a single platform. Key innovative aspects include:

- Delayed AI guidance that promotes deep thinking before assistance.
- Pattern-based curriculum design aligning with cognitive learning models.
- Explainable AI debugging for clarity and error reasoning.
- Institutional proctoring and anti-plagiarism integration.This multi-faceted innovation positions CodeMitra as a next-generation edtech tool for adaptive and ethical coding education.

1.6.3 Usefulness

CodeMitra is beneficial across multiple stakeholders:

- For Students: It strengthens conceptual understanding, encourages independent problem-solving , and provides real-time personalized feedback.
- For Educators: It automates grading, identifies weak learning areas, and generates analytical insights for targeted interventions.
- For Institutions: It ensures exam integrity, reduces manual workload, and supports large-scale deployment for fair assessments.Thus, the system contributes significantly to the improvement of learning outcomes and institutional credibility

1.7 Report Organization

The remainder of this report is structured as follows:

- Chapter 2 : Discusses existing tools, research studies, and methodologies related to AI-assisted learning platforms, coding education, plagiarism detection, and

secure assessments. It highlights current trends and identifies research gaps that motivated the development of CodeMitra.

- Chapter 3 : Defines the functional and non-functional requirements of the system, detailing user needs, system scope, feasibility analysis, and requirement specification models. This chapter establishes the foundation for the system's design and implementation.
- Chapter 4 : Presents the architectural framework, data flow diagrams, UML models, interface design, and database schema of CodeMitra. It explains how different modules interact and how the system achieves modularity, scalability, and robustness.
- Chapter 5 : Summarizes the key outcomes of the project, reflecting on how the objectives were achieved. It also outlines potential future enhancements, such as incorporating advanced AI models, adaptive learning analytics, and expanded institutional deployment capabilities.
- References and Appendices : Include all scholarly works, online sources, and supplementary materials (e.g., screenshots, test cases, or survey results) that support the project and its evaluation.

REVIEW OF LITERATURE

2.1 Preliminary Investigation

A preliminary study was conducted to review major coding platforms and AI tools such as LeetCode, HackerRank, and GitHub Copilot, as well as research literature related to automated assessment and programming education.The main goal was to understand how current systems support learning and identify their limitations.

Key findings include:

- Most platforms provide problem-solving environments but lack pattern-based learning structures that guide students through different solution stages.
- AI tools like Copilot increase productivity but can make learners dependent instead of encouraging independent reasoning.
- Automated grading systems work well for correctness checking but often fail to ensure originality or academic integrity.
- Feedback on most platforms is limited to “pass/fail” outputs, offering little conceptual explanation or debugging support.

2.1.1 Current System

LeetCode and HackerRank are widely used for competitive programming and technical interview preparation. They provide extensive problem sets and automatic judging systems, but they do not support stepwise learning progression or guided feedback. Students are mainly evaluated based on output correctness.

GitHub Copilot, an AI-based code generation tool, assists developers by suggesting or completing code snippets. While it helps improve productivity, it does not promote structured learning or conceptual understanding. Moreover, it lacks institutional features such as user monitoring, reporting, and plagiarism prevention.

From these observations, it is evident that current systems focus more on practice and productivity rather than learning process, reasoning, and secure assessment — creating a gap that CodeMitra aims to fill.

2.2 Limitations of Current System

Based on the investigation, several limitations in the existing systems have been identified:

1. No Progressive Learning Support: Learners are not guided through stages like *Brute Force → Improved → Optimal*, resulting in shallow understanding of algorithmic concepts.
2. Limited Feedback and Explainability: Current platforms give test-case results but rarely explain the reasoning behind errors or suggest improvements.
3. Weak Anti-Cheating and Institutional Control: Most systems are designed for individuals, with no proctoring, plagiarism detection, or secure assessment.

2.3 Chapter Summary

Table-01: 2.3.1: Chapter Summary Table

Author/ Source		Year	Focus	Limitations	Relevance to CodeMitra
LeetCode Platform		2015–Present	Online problem-solving and judging platform.	Lacks guided learning or feedback explainability.	Highlights the need for progressive and pattern-based learning.
HackerRank Platform		2014–Present	Coding challenges and interview preparation.	Focuses on correctness; weak conceptual guidance.	Supports the idea of AI-based conceptual feedback.
GitHub Copilot		2021	AI code suggestion tool using large language models.	Encourages code dependency; lacks institutional use.	Inspires controlled AI assistance in CodeMitra.
Patel et al., IJCA		2019	Automated coding assessment review.	Weak plagiarism control; limited scalability.	Motivates inclusion of anti-copy and proctoring measures.

REQUIREMENT ENGINEERING

3.1 Feasiblity Study (Technical, Economical, Operational)

Technical Feasibility:

The proposed system, *CodeMitra*, is technically feasible with current technologies. The integration of Groq API for AI-driven hint generation, Piston API for secure code execution, and a React + Spring Boot architecture ensures modularity and scalability. The backend framework (Spring Boot) can handle high concurrency, while the frontend offers a responsive, modern interface. Deployment on AWS Cloud (EC2 for computation, RDS for database, S3 for storage) ensures reliable uptime, scalability, and ease of maintenance.

Economic Feasibility:

The project involves moderate development and operational costs primarily related to cloud hosting, API subscriptions, and content curation. These costs can be easily offset through institutional subscriptions, student premium plans, or partnerships with coding academies. Since the platform promotes institutional usage, the return on investment is expected to be high within a short operational period.

Operational Feasibility:

Operationally, the system requires a small but dedicated content and admin team to curate coding problems, verify test patterns, and manage institutional accounts. AI-assisted hint generation reduces manual workload, while integrated analytics streamline evaluation and reporting. Therefore, the system is operationally sustainable with minimal human supervision once established.

3.2 Requirement Collection

The requirements for CodeMitra were collected through **surveys and interviews** with students, instructors, and academic administrators.

3.2.1 Discussion

Stakeholders:

- **Students** – Require interactive problem-solving, hints, and feedback.
- **Teachers** – Need reliable testing, plagiarism detection, and grading automation.
- **Administrators** – Focus on institutional control, reports, and security compliance.

Priority Features:
Secure institutional tests, AI-powered debugging and hints, automated grading, and gamified scoring systems emerged as the top priorities.

3.2.2 Requirement Analysis

Functional Priorities:

- Problem browsing and code submission.
- Integrated code editor with execution environment.
- Contextual hinting and error explanations via AI.
- Institutional test creation and automated grading.
- Leaderboards and analytics dashboards.

Nonfunctional Priorities:

- **Security:** Authentication, data encryption, secure API communication.
- **Performance:** System latency under 2 seconds for UI actions.
- **Scalability:** Cloud-native design to handle large test sessions.
- **Availability:** Minimum uptime of 99.5%.

3.3 Requirements

3.3.1 Functional Requirements

Table-02 : 3.3.1.1 : Statement of Functionality

ID	Functional Requirement
FR1	Users can register, authenticate, and manage personal profiles securely.
FR2	Students can attempt coding problems using an integrated code editor and execute code in real-time.
FR3	The system provides time-gated AI hints and explainable debug suggestions based on code errors.
FR4	Administrators can create, schedule, and monitor institutional tests with anti-cheating measures.
FR5	Auto-grading evaluates submissions for both correctness and efficiency, generating detailed performance reports.

3.3.2 Nonfunctional Requirments

Table-03 : 3.3.2.1 Statement of Functionality

ID	Nonfunctional Requirement
NFR1	Data security ensured via TLS encryption and hashed passwords.
NFR2	Average response time for code execution under 5 seconds.
NFR3	Supports concurrent sessions for over 1000 users.
NFR4	Intuitive and responsive user interface ensuring accessibility.

3.4 Hardware & Software Requirements

Table-04 : 3.4.1 Hardware Requirement (Developer & End User)

UserType	Specification
Developer	Intel i5 / AMD Ryzen processor, 16GB+ RAM, SSD storage, optional GPU for local ML testing.
End User	Any modern device (PC, laptop, or tablet) with a web browser and at least 4GB RAM.

Table-05 : 3.4.2 Software Requirement (Developer & End User)

Component	Technology Stack
Frontend	HTML , JavaScript , CSS , React.js .
Backend	Java, Spring Boot, RESTful APIs.
Database	MySQL.
AI Integration	Groq API for intelligent hint generation.
Code Execution	Piston API sandbox.

Cloud Services	AWS EC2 (compute), RDS (database), S3 (storage).

3.5 Use-case Diagrams

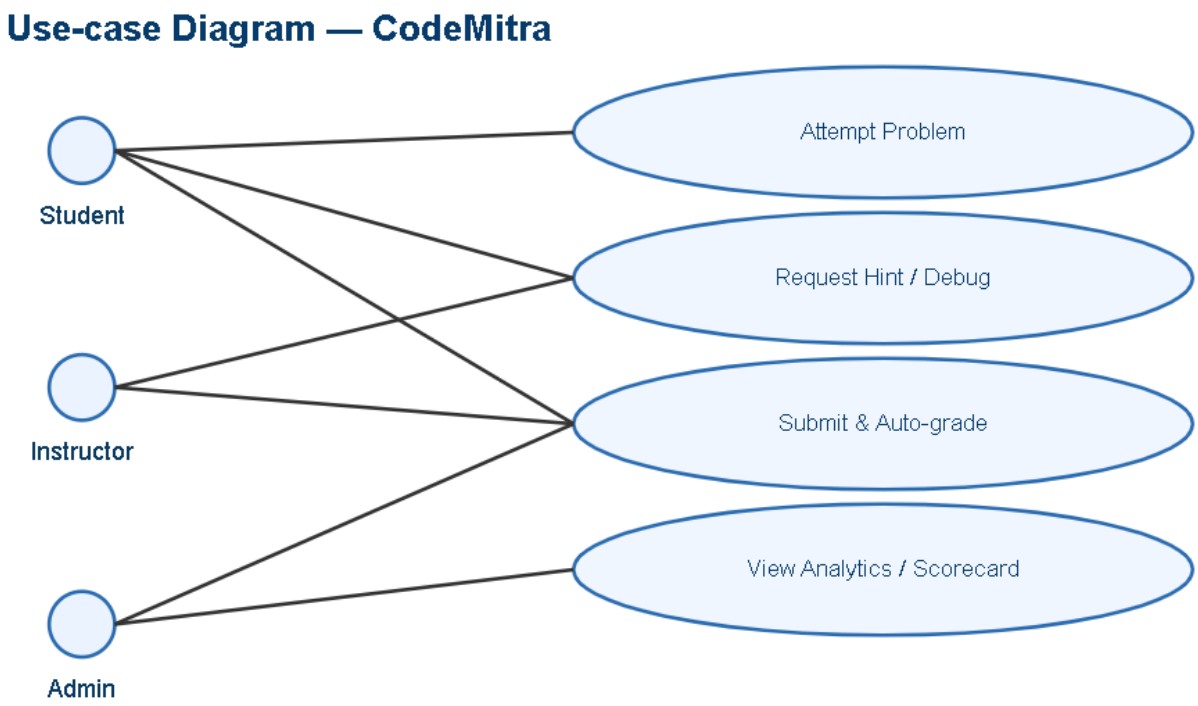

Figure : 01

3.5.1 Use-Case Descriptions

The primary use cases for CodeMitra include:

- **Attempt Problem:** Student browses and solves coding problems in the integrated editor.
- **Request Hint:** AI provides contextual, time-gated guidance.
- **Submit Code:** Code is auto-evaluated for correctness and efficiency.
- **Create Test (Admin):** Admin configures tests with anti-cheating measures and monitors progress.
- **View Reports:** Teachers and administrators access analytical insights on performance.

Analysis & Conceptual Design & Technical Architecture

4.1 Technical Architecture

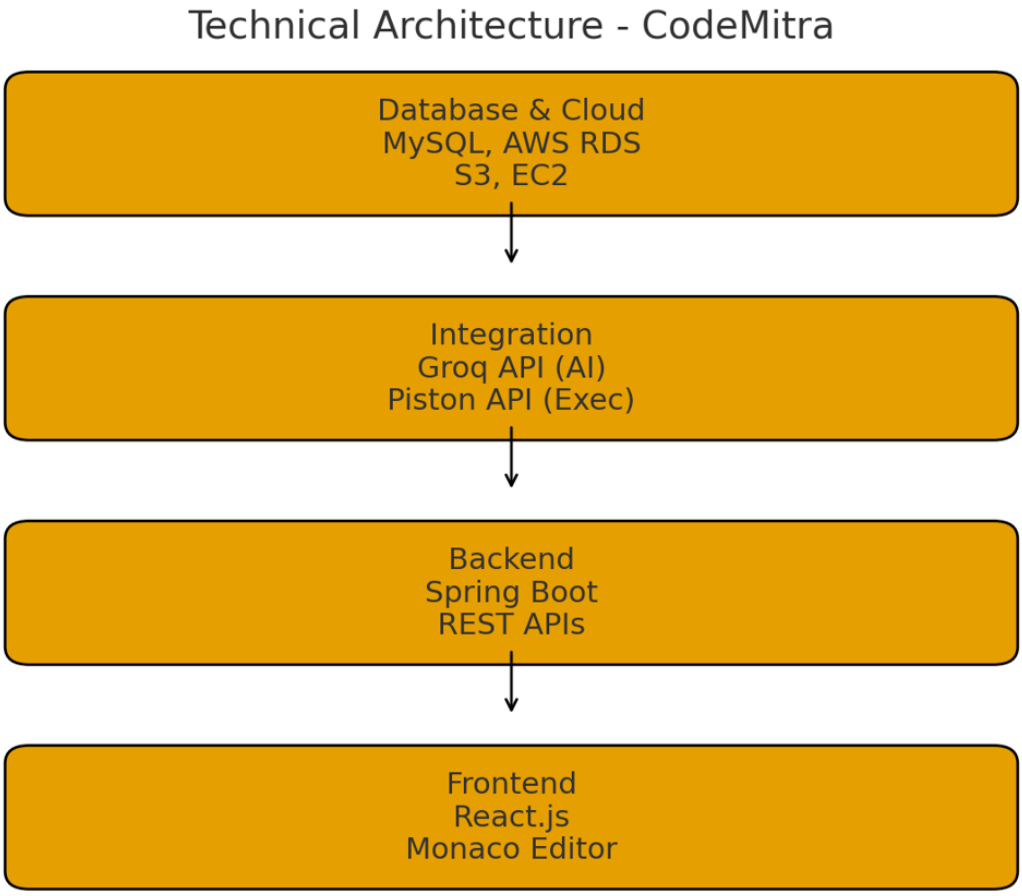

Figure : 02

Description:

CodeMitra follows a **three-tier cloud-based architecture** integrating AI, code execution, and institutional testing.

Layers:

1. **Presentation Layer (Frontend)** – React.js UI with Monaco Editor for coding, dashboards for users and admins.
2. **Application Layer (Backend)** – Spring Boot APIs managing authentication, test management, submissions, and analytics.
3. **Integration Layer** – Connects with external APIs:
   - **Groq API:** For AI-generated hints and debugging.
   - **Piston API:** For code execution sandboxing.

4. **Database Layer** – MySQL for users, submissions, and institutional data.
5. **Cloud Infrastructure** – AWS (EC2, RDS, S3) for hosting and scalability.

4.2 Sequence Diagrams

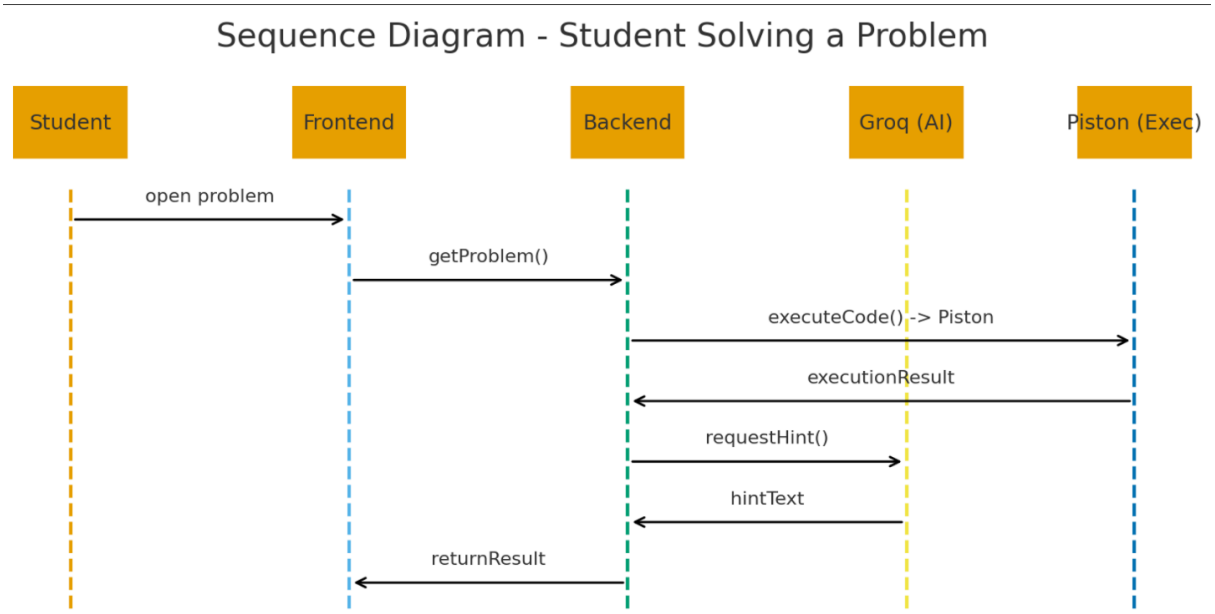

Figure : 03

Use Case 1: Student Solving a Problem

Actors: Student, System (Frontend + Backend), AI Engine (Groq), Code Executor (Piston)

Flow:

1. Student logs in.
2. Student selects a problem.
3. Frontend requests problem details from backend.
4. Student writes code and submits for execution.
5. Backend sends code to Piston API → returns output.
6. If error occurs, student requests hint.
7. Backend calls Groq API → returns contextual AI hint.
8. Result and hint are displayed to the user.

4.3 Class Diagrams

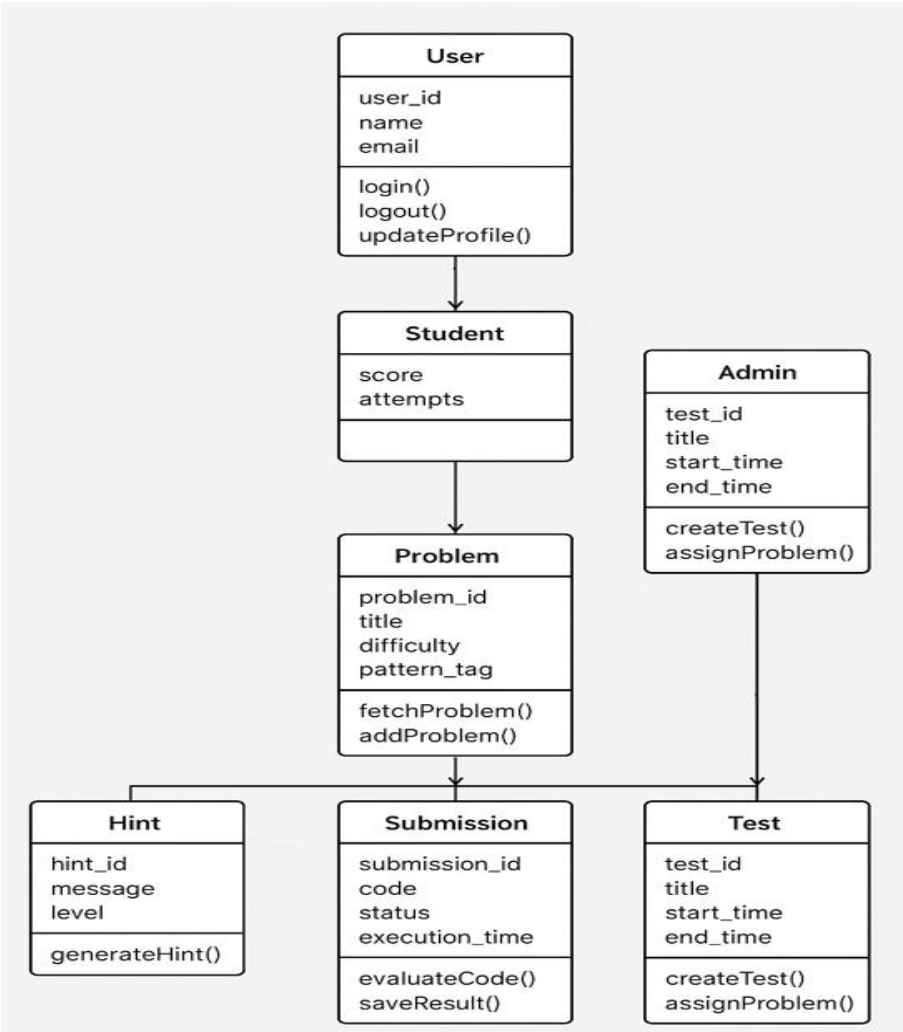

Figure : 04

Main Classes and Relationships:

- **User**
  - Attributes: userID, name, email, role, passwordHash
  - Methods: login(), logout(), updateProfile()
- **Problem**
  - Attributes: problemID, title, difficulty, patternTag, description
  - Methods: fetchProblem(), addProblem()
- **Submission**

    - Attributes: submissionID, userID, problemID, code, result, timestamp
    - Methods: evaluateCode(), saveResult()
- **HintEngine**
    - Attributes: hintID, problemID, hintText, delayTime
    - Methods: generateHint(), fetchHint()
- **Test**
    - Attributes: testID, adminID, testName, startTime, endTime
    - Methods: createTest(), assignProblems()
- **Report**
    - Attributes: reportID, userID, testID, score, accuracy
    - Methods: generateReport(), viewReport()

Relationships:

- User → Submission (1:N)
- Problem → Submission (1:N)
- Admin (User subtype) → Test (1:N)
- Test → Problem (N:M)
- Submission → Report (1:1)

4.4 DFD

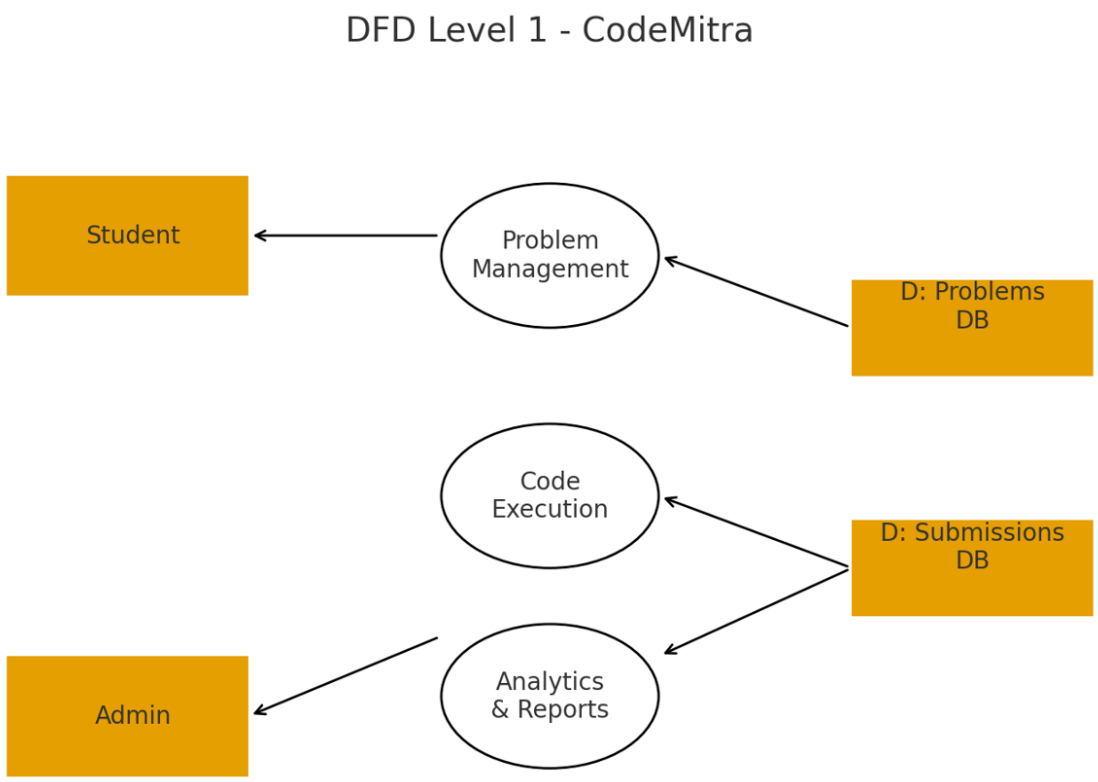

Figure : 05

Level 0 (Context Diagram):
Entities:

- Student
- Teacher/Admin
- System (CodeMitra)

Process:
Students and teachers interact with CodeMitra. The system processes login, problem solving, hint generation, and reporting.
Level 1 (Main Processes):

1. **User Authentication** – validate credentials, manage sessions.
2. **Problem Management** – retrieve problems, store solutions.
3. **Code Execution** – send code to Piston API, return results.
4. **Hint Generation** – request AI hints from Groq API.
5. **Test Management** – create, assign, and evaluate tests.
6. **Analytics and Reporting** – generate performance dashboards.

Data Stores:

- D1: User Database
- D2: Problem Bank
- D3: Submission Records
- D4: Reports and Analytics

4.5 User Interface Design

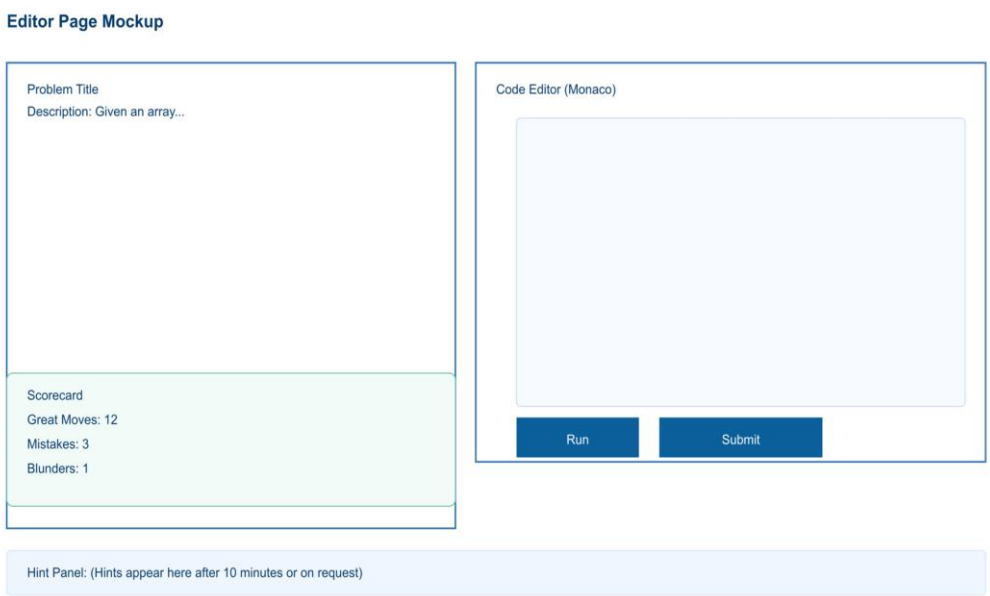

Figure : 06

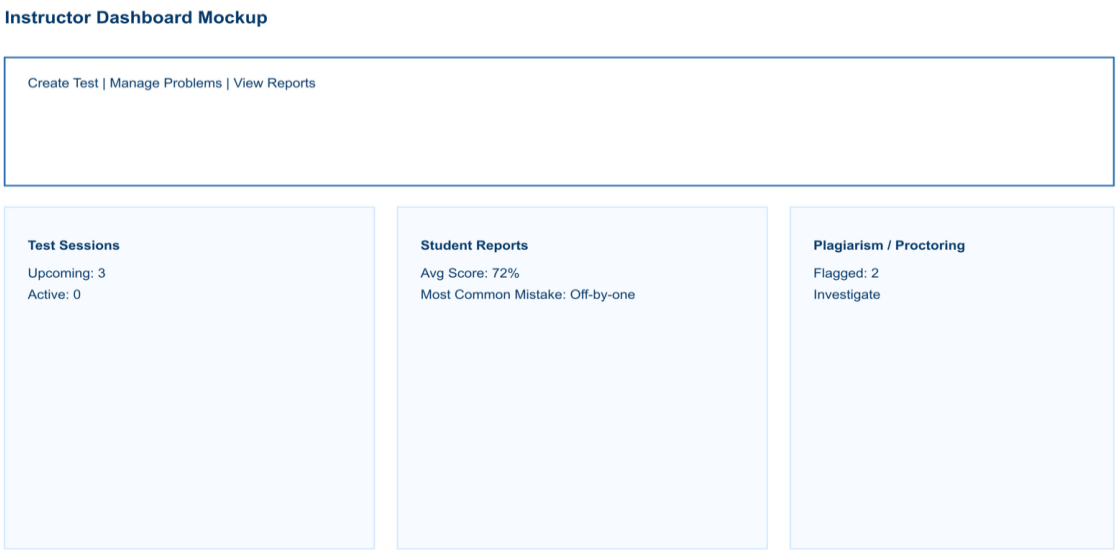

Figure : 07

4.6 Data Design

Table-06 : 4.6.1 Schema Definitions

Table	Key Fields	Description
Users	user_id (PK), name, email, password_hash, role	Stores student/admin details.
Problems	problem_id (PK), title, pattern, difficulty, description	Contains coding problem details.
Submissions	submission_id (PK), user_id (FK), problem_id (FK), code, result, timestamp	Stores user code submissions and outputs.
Tests	test_id (PK), admin_id (FK), test_name, start_time, end_time	Holds institutional test details.
Reports	report_id (PK), user_id (FK), test_id (FK), score, accuracy	Stores test performance data.

4.6.2 E-R Diagram

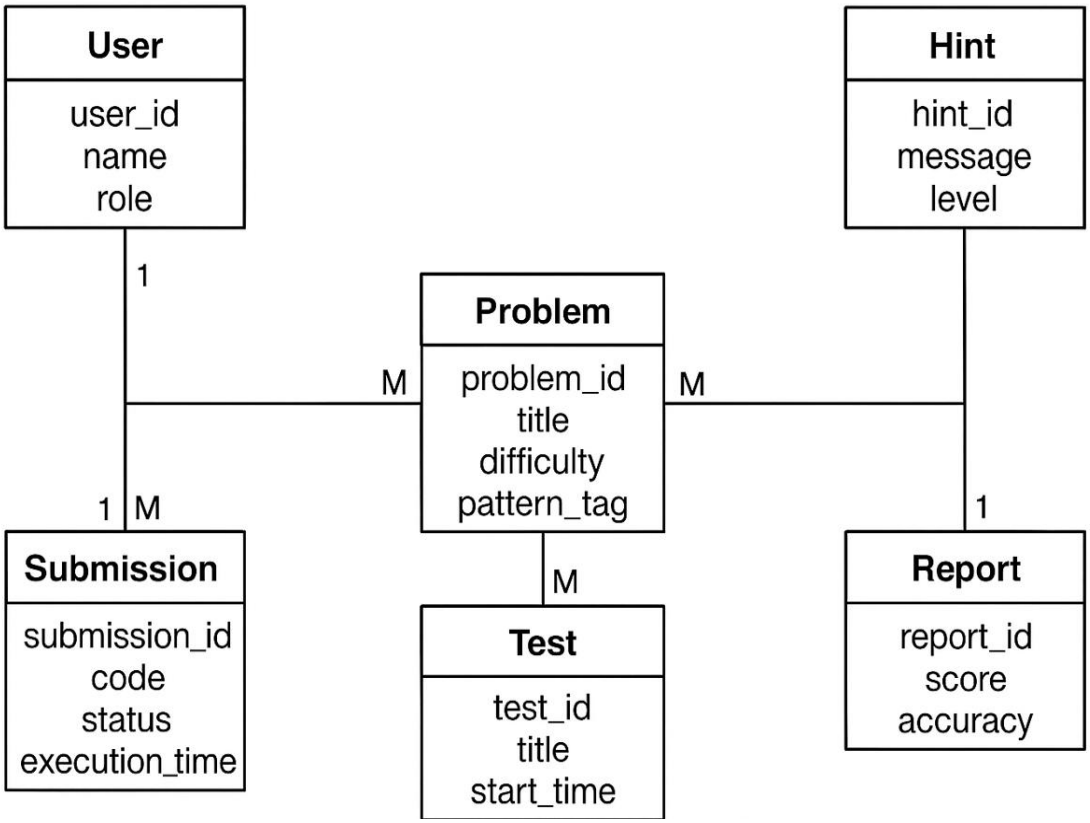

Figure : 08

Entities & Relationships:

- **User** (UserID, Name, Email, Role)
  ↳ *submits* → **Submission**
- **Problem** (ProblemID, Title, Pattern, Difficulty)
  ↳ *included in* → **Test**
- **Test** (TestID, TestName, Schedule)
  ↳ *has* → **Report**
- **Report** (ReportID, Score, Accuracy)
  ↳ *belongs to* → **User**
- **HintEngine** (HintID, ProblemID, HintText)
  ↳ *generates hints for* → **Problem**

Relationships:

- User : Submission → 1:N
- Problem : Submission → 1:N
- Admin (User) : Test → 1:N
- Test : Problem → N:M
- Submission : Report → 1:1

Conclusion & Future Scope

5.1 Conclusion

The development of **CodeMitra** successfully bridges the gap between conventional coding practice platforms and pedagogically enriched learning environments. Unlike traditional systems that focus solely on correctness and leaderboard performance, CodeMitra emphasizes a **pattern-based learning approach** that helps students internalize core problem-solving templates (Brute → Improved → Optimal). The integration of **AI-driven contextual guidance** and **explainable debugging** empowers learners to understand their mistakes and gradually enhance their reasoning skills instead of relying on rote memorization or code copying.

For institutions, the platform provides a **secure and scalable assessment framework** equipped with anti-plagiarism, proctoring, and analytics modules. This not only ensures fair evaluation but also allows educators to monitor conceptual progress through detailed performance metrics. The combination of **AI assistance, pedagogical structuring, and robust security** positions CodeMitra as a holistic and innovative solution for modern coding education.

In summary, CodeMitra contributes meaningfully to improving coding pedagogy, reducing plagiarism, and providing adaptive support to learners and institutions alike — making it a significant step toward intelligent and ethical programming education.

5.2 Future Scope

- Expand problem repository and pattern taxonomy.
- Add personalized curricula using learner modeling and reinforcement learning.
- More advanced proctoring (AI-based behavior analysis) and improved plagiarism detectors using code similarity metrics and stylometry.
- Mobile app for on-the-go practice and offline hints caching.
- Multi-language support and community problem-creation workflows.

REFERENCES

- Groq, "AI-Powered Code Understanding and Learning," Online. Available: www.groq.com. Accessed: Oct. 2025.
- LeetCode, "Coding Practice Platform – Problems and Solutions," Online. Available: www.leetcode.com. Accessed: Oct. 2025.
- HackerRank, "Interview Preparation Kit and Coding Challenges," Online. Available: www.hackerrank.com. Accessed: Oct. 2025.
- GitHub Copilot, "AI Pair Programming Tool," Online. Available: www.github.com/features/copilot. Accessed: Oct. 2025.
- Piston API, "Online Code Execution Engine," Online. Available: www.piston.rs. Accessed:Oct.2025.
- M. Gupta and R. Verma, "Security Measures in Digital Coding Platforms," International Journal of Technology in Education, vol. 14, no. 3, pp. 56–64, 2020.
- P. Kumar et al., "Designing User-Friendly Interfaces for Coding Applications," Computers and Multimedia in Education, vol. 162, pp. 123–133, 2019.
- Stack Overflow, "Learning Programming Patterns and Best Practices," Online. Available: www.stackoverflow.com. Accessed: Oct. 2025.
- GitHub, "Code Repositories and Learning Resources," Online. 3Available: www.github.com. Accessed: Oct. 2025.
- Coursera, "AI-Assisted Learning and Intelligent Tutoring Systems," Online. Available: www.coursera.org. Accessed: Oct. 2025.
- Google Cloud, "Scalable Cloud Architecture for Modern Web Applications," Online. Available: cloud.google.com. Accessed: Oct. 2025.
- JetBrains, "IntelliJ IDEA Debugger and Code Analysis Tools," Online. Available: www.jetbrains.com. Accessed: Oct. 2025.
- GeeksforGeeks, "Programming Patterns and Data Structures Tutorials," Online. Available: www.geeksforgeeks.org. Accessed: Oct. 2025.
- E. Anderson et al., "Gamification Techniques in Online Learning Platforms," International Journal of Educational Technology, vol. 18, no. 2, pp. 44–57, 2021.
- T. Johnson and S. Patel, "AI-Based Automated Code Assessment Systems," Journal of Computer Science Education, vol. 23, no. 1, pp. 11–23, 2022